Electric Power Systems Research 213 (2022) 108566

Contents lists available at ScienceDirect

Electric Power Systems Research

journal homepage: www.elsevier.com/locate/epsr

Learning optimization proxies for large-scale Security-Constrained Economic Dispatch

Wenbo Chen, Seonho Park, Mathieu Tanneau *, Pascal Van Hentenryck

Georgia Institute of Technology, United States of America

ARTICLE INFO

Keywords:
Security-constrained economic dispatch
Optimization proxy
Deep learning
Neural network

ABSTRACT

The Security-Constrained Economic Dispatch (SCED) is a fundamental optimization model for Transmission System Operators (TSO) to clear real-time energy markets while ensuring reliable operations of power grids. In a context of growing operational uncertainty, due to increased penetration of renewable generators and distributed energy resources, operators must continuously monitor risk in real-time, i.e., they must quickly assess the system's behavior under various changes in load and renewable production. Unfortunately, systematically solving an optimization problem for each such scenario is not practical given the tight constraints of real-time operations. To overcome this limitation, this paper proposes to learn an optimization proxy for SCED, i.e., a Machine Learning (ML) model that can predict an optimal solution for SCED in milliseconds. Motivated by a principled analysis of the market-clearing optimizations of MISO, the paper proposes a novel just-in-time ML pipeline that addresses the main challenges of learning SCED solutions, i.e., the variability in load, renewable output and production costs, as well as the combinatorial structure of commitment decisions. A novel combined classification-plus-regression architecture is also proposed, to further capture the behavior of SCED solutions. Numerical experiments are reported on the French transmission system, and demonstrate the approach's ability to produce, within a time frame that is compatible with real-time operations, accurate optimization proxies that produce relative errors below 1%.

1. Introduction

The *Security-Constrained Economic Dispatch* (SCED) is a fundamental optimization model for Transmission System Operators (TSOs) to clear real-time energy markets while ensuring reliable operations of power grids [1]. In the US, TSOs like MISO and PJM execute a SCED every five minutes, which means that the optimization problem must be solved in an even tighter time frame, i.e., well under a minute [2]. Security constraints, which enforce robustness against the loss of any individual component, render SCED models particularly challenging for large systems [3–5] unless only a subset of contingencies is considered. With more distributed resources and increased operational uncertainty, such computational bottlenecks will only become more critical [2].

This paper is motivated by the growing share of renewable generation, especially wind, in the MISO system, which calls for risk-aware market-clearing algorithms. One particular challenge is the desire to perform risk analysis in real time, by solving given a large number of scenarios for load and renewable production [6]. However, systematically solving many SCED instances is not practical given the tight constraints of real-time operations. To overcome this computational challenge, this paper proposes to learn an optimization proxy for SCED, i.e., a Machine Learning (ML) model that can predict an optimal solution for SCED, within acceptable numerical tolerances and in milliseconds.

It is important to emphasize that the present goal is not to replace optimization-based market-clearing tools. Instead, the proposed optimization proxy provides operators with an additional tool for risk assessment. This allows to quickly evaluate how the system would behave under various scenarios, without the need to run costly optimizations. In particular, because these predictions are combined into aggregated risk metrics, small prediction errors on individual instances are acceptable.

1.1. Related literature and challenges

The combination of ML and optimization for power systems has attracted increased attention in recent years. A first thread [7–14] uses ML models to predict an optimal solution to the Optimal Power Flow

* Corresponding author.
E-mail addresses: wenbo.chen@gatech.edu (W. Chen), spark719@gatech.edu (S. Park), mathieu.tanneau@gatech.edu (M. Tanneau), pascal.vanhentenryck@isye.gatech.edu (P. Van Hentenryck).

https://doi.org/10.1016/j.epsr.2022.108566
Received 4 October 2021; Received in revised form 17 April 2022; Accepted 2 July 2022
Available online 2 August 2022

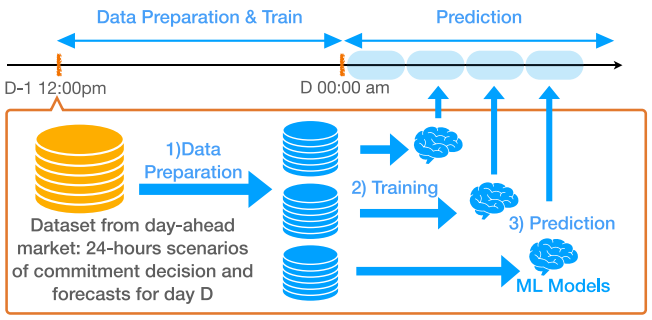

Fig. 1. The machine learning pipeline.

(OPF) problem. Pan et al. [7] train a Deep Neural Network (DNN) model to predict solutions to the security-constrained DC-OPFs, and report results on systems with no more than 300 buses. One common limitation of ML models, also noted in [7], is that predictions are not guaranteed to satisfy the original problem's constraints. To address this limitation, recent efforts [8,9] integrate Lagrangian duality into the training of DNNs in order to capture the physical and operational constraints of AC-OPFs. A similar approach is followed in Velloso et al. [12] in the context of preventive security-constrained DC-OPF. Namely, a DNN is trained using Lagrangian duality techniques, then used as a proxy for the time-consuming master problem in a column-and-constraint generation algorithm. More recently, Chatzos et al. [11] embed the Lagrangian duality framework in a two-stage learning that exploits a regional decomposition of the power network, enabling a more efficient distributed training.

Another research thread is the integration of ML models within optimization algorithms, in order to improve runtime performance. For instance, a number of papers (e.g., [15–19]) try to identify active constraints in order to reduce the problem complexity. In [17], the authors investigate learning feasible solutions to the unit commitment problem. Venzka el al. [20] use a DNN model to learn the feasible region of a dynamic SCOPF and transform the resulting DNN into a mixed-inter linear program.

Existing research papers in ML for power systems typically suffer from two limitations. On the one hand, most papers report numerical results on small academic test systems, which are one to two orders of magnitude smaller than real transmission systems. This is especially concerning, as higher-dimensional data has an adverse impact on convergence and accuracy of machine-learning algorithms. On the other hand, almost all papers rely on artificially-generated data whose distribution does not capture the variability found in actual operations. For instance, only changes in load are considered, typically without capturing spatio-temporal correlations. Other sources of uncertainty, such as renewable production, are not considered, nor is the variability of economic bids. Finally, changes in commitment decisions throughout the day are rarely addressed, although they introduce non-trivial, combinatorial, distribution shifts.

1.2. Contributions and outline

To address the above limitations, the paper proposes a novel ML pipeline that is grounded in the structure of real-world market operations. The approach leverages the TSO's forward knowledge of (1) commitment decisions and (2) day-ahead forecasts for load and renewable productions. Indeed, both are available by the time the day-ahead market has been executed. Furthermore, the paper presents an in-depth analysis of the real-time market behavior, which informs a novel ML architecture to better captures the nature of actual operations.

The proposed learning pipeline is depicted in Fig. 1. First, commitment decisions and day-ahead forecasts for load and renewable generation are gathered, following the clearing of the day-ahead market. Second, this information is used to generate a training dataset of SCED instances, using classical data augmentation techniques. Third, specialized ML models are trained, ideally one for each hour of the operating day, thereby alleviating the combinatorial explosion of commitment decisions: each ML model only needs to consider one set of commitments and focus on load/renewable scenarios around the forecasts for that hour. Fourth, throughout the operating day, the trained models are used in real time to evaluate large number of scenarios. The entire data-generation and training procedure is completed in a few hours.

The rest of the paper is organized as follows. Section 2 describes the interplay between day-ahead and real-time markets in the MISO system, and gives an overview of the real-time SCED for MISO. Section 3 analyzes the behavior of the real-time market solutions, proposes a combined classification-plus-regression architecture, and presents the overall ML pipeline. Numerical experiments on a real-life system are reported in Section 4

2. Overview of MISO's market-clearing pipeline

This section describes the interplay between MISO's day-ahead and real-time markets; the reader is referred to [21] for a detailed overview of MISO's energy and reserve markets. While the paper focuses on MISO's market-clearing pipeline, other ISOs in the US also operate day-ahead and real-time markets. These include, e.g., CAISO [22], ISO-NE [23], PJM [24], and NYISO [25]. Therefore, the paper's proposed methods easily extend to other US ISOs.

2.1. The MISO optimization pipeline

The day-ahead market consists of two phases, and is executed every day at 10am. First, a day-ahead security-constrained unit commitment (DA-SCUC) is executed: it outputs the commitment and regulation status of each generator for every hour of the following day. Then, a day-ahead SCED is executed to compute day-ahead prices and settle the market. The results of the day-ahead market clearing are then posted online at approximately 1pm, i.e., there is a delay of several hours before the commitment decisions take effect. While MISO may commit additional units during the operating day, through out-of-market reliability studies, in practice, 99% of commitment decisions are decided in the day-ahead market [2]. Accordingly, for simplicity and without loss of generality, this paper assumes that commitment decisions from the DA-SCUC are not modified afterwards.

Then, throughout the operating day, the real-time market is executed every 5 min. This real-time SCED (RT-SCED) adjusts the dispatch of every generator in response to variations in load and renewable production and maximizes economic benefit. Despite its short-time horizon, the RT-SCED must still account for uncertainty in load and renewable production. In the MISO system, this uncertainty mainly stems from the intermittency of wind farms, and from increasing variability in load. The latter is caused, in part, by the growing number of behind-the-meter distributed energy resources (DERs) such as residential storage and rooftop solar. Therefore, operators continuously monitor the state of the power grid, and may take preventive and/or corrective actions to ensure safe and reliable operations.

2.2. The RT-SCED formulation

The MISO RT-SCED uses a DC-based formulation to co-optimize energy and reserves [26]. Due to space limitations (the full formulation is a 220-page document), only its core elements are described here; a simplified formulation is stated in [27]. The RT-SCED is a linear program of the form

$$\min_{p,r,z} \left\{ c^T p + d^T r + q^T z \mid Ap + Br + Mz \geq b \right\}, \tag{1}$$

where p denote the active power dispatch variables, r denotes the reserve dispatch variables, and z denotes additional variables that capture constraint violations. Line losses are estimated in real-time from MISO's state estimator, and incorporated in the formulation using a loss factors approach as in [28]. Transmission constraints are modeled using PTDF matrices, where flow sensitivities with respect to power injections and withdrawals are provided by an external tool. Transmission limits are soft, i.e., they may be violated, albeit at a reasonably high cost. Each generator is subject to ramping constraints and individual limits on energy and reserve dispatch. Production costs are modeled as piecewise linear convex functions. Each market participant submits its own production costs via MISO's market portal, and may submit different offers for each hour of the day. For intermittent generators such as solar and wind, the latest forecast is used in lieu of binding offers. The RT-SCED also considers our categories of reserves: regulating, spinning, online supplemental, and offline supplemental. Reserves are dispatched on individual generators, in order to meet zonal and market-wide minimum requirements. Additional constraints ensure that, in a contingency event, reserves may be deployed without tripping transmission lines. Both line and generator contingencies are considered.

In summary, the RT-SCED receives the following inputs: the commitment decisions from DA-SCUC, the most recent forecast for load and renewable production, the economic limits and production costs of the generators, the current state estimation, and the transmission constraints and reserve requirements. The RT-SCED produces as outputs the active power p and reserve dispatch r for each generator.

3. The learning methodology

This section reviews the learning methodology for the RT-SCED. First, Section 3.1 presents an analysis of the behavior of optimal SCED solutions in MISO's optimization pipeline. These patterns motivate a novel ML architecture, which is described in Section 3.2, followed by an overview of the proposed ML pipeline in Section 3.3. Further details on the data are given in Section 4, as wells as in [29].

3.1. Pattern analysis of optimal SCED solutions

The system at hand is the French transmission system, whose network topology is provided by the French TSO, RTE. It contains approximately 6,500 buses, 9,000 transmission lines and transformers, and 1,800 individual generators, 1,100 of which are wind and solar generators and 700 are conventional units. The MISO optimization pipeline, described in Section 2, is replicated on this system for the entire year 2018, yielding 365 DA-SCUC instances and about 100,000 RT-SCED instances. Relevant statistics are reported next.

First, unsurprisingly, commitment decisions display a high variability, even within a single day. For instance, in January, the median distance between two hourly commitment decisions in a given day, is about 70 generators, i.e., 10% of the conventional units may see different commitments throughout the day. For February, this number rises to about 120 generators, or 18% of conventional units. This variability is on average twice smaller during the spring and summer months, namely, April–September. The intra-day variability of commitment decisions is illustrated in Fig. 2. Namely, for each month of the year, Fig. 2 displays the distribution, across every day of the month, of the number of unique hourly commitments over a day. The higher values correspond to the higher variability in commitment decisions, while the lower values indicate that the commitment decisions are stable throughout the day. Typically, the variability of commitment decisions is lower in summer and higher in winter; this behavior is expected since more generators are online in winter, which naturally tends to yield more diverse commitments. Indeed, in June 2018, all the days have at least 5 and at most 19 different hourly commitments, with 50% of days having less than 7 and 50% having more than 8. In contrast, in January 2018, except for two outliers, every day

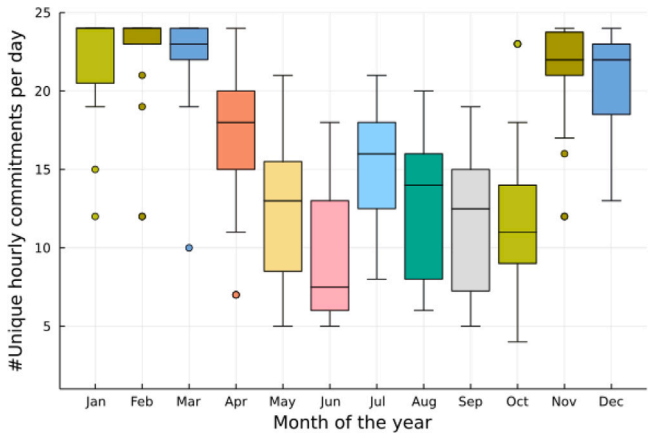

Fig. 2. Distribution of unique hourly commitments per day, for each month of the year 2018. Higher values indicate higher variability in commitment decisions.

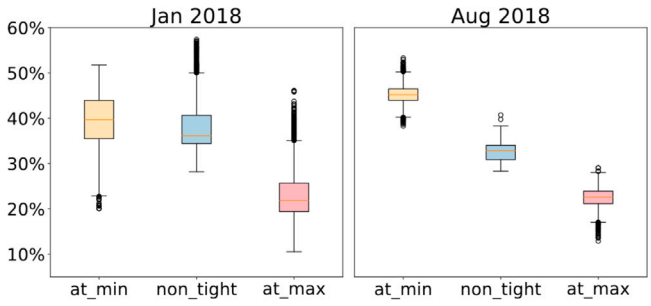

Fig. 3. Proportion of generators at their maximum and minimum limits in January (left) and August (right).

has at least 19 different commitments, and 16 days had a different commitment decision every hour. Overall this combinatorial explosion of commitment decisions has an adverse effect on ML models, since it creates distribution shifts on unseen commitments that complicate the learning task.

Second, an analysis of RT-SCED solutions reveals that a majority of generators are dispatched to either their minimum or maximum achievable limit. Such limits include economic offer data and ramping constraints, and have the form

$$\max(P_g^{\min}, p_g^0 - R_g^{\mathrm{dn}}) \le p_g \le \min(P_g^{\max}, p_g^0 + R_g^{\mathrm{up}}), \tag{2}$$

where, for generator g, p_g is the active power dispatch variable, p_g^0 is the initial output (from previous SCED solution), $P_g^{\min}, P_g^{\max}$ are the economic minimum and maximum limits, and $R_g^{\mathrm{dn}}, R_g^{\mathrm{up}}$ are the 5-minute ramping-down and -up capacities. Detailed statistics are reported in Fig. 3 for January and August 2018: each plot quantifies, across all RT-SCED instances for that month, the proportion of generators for which constraint (2) is binding at the lower-bound (`at_min`), upper-bound (`at_max`), or non-binding (`non_tight`). These statistics exclude generators for which the minimum and maximum limit are identical. In January, the median proportion of generators being dispatched at their minimum (resp. maximum) limit is close to 20% (resp. 60%), while in August, these values are around 20% and 80%. The variability is also visibly higher in winter, echoing the previous observations for commitment decisions.

3.2. First classify, then perform regression

The previous analysis indicates that, given the knowledge of which generators are dispatched to their minimum or maximum limit, only a small number of generators need to be predicted. This suggests a

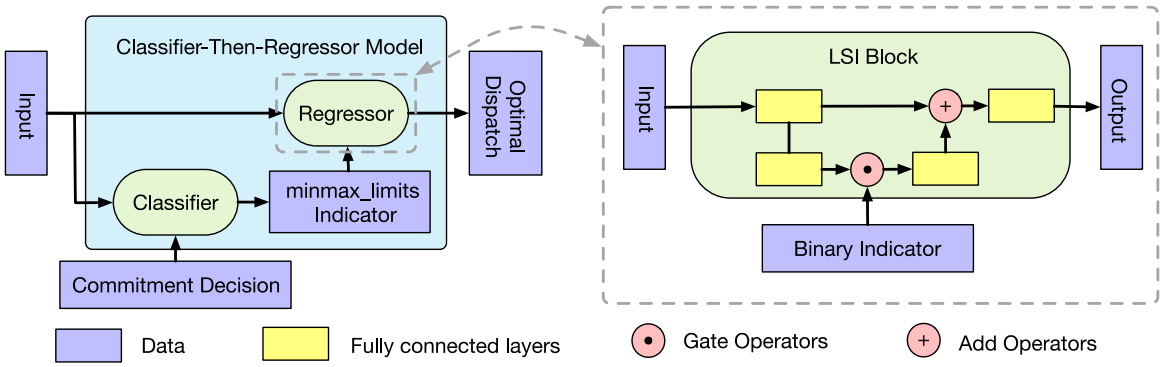

Fig. 4. The proposed ML model: (1) the figure on the left shows the structure of the Classifier-Then-Regressor model, where the outputs of the classifier are used to inform the subsequent regressor. Thus the regressor only needs to predict the dispatches of non-tight generators; (2) the figure on the right shows the structure of the regressor, i.e., a LSI block, where a gate operation takes the binary indicator to dropout some neurons of the previous fully connected layer.

Table 1
Input features of DNN model.

Feature	Size	Source
Loads	l	Load forecasts
Cost of generators	g	Bids
Cost of reserves	$2g$	Bids
Previous solution	g	Solver
Commitment decisions	g	SCUC
Reserve Commitments	g	SCUC
Generator min/max limits	$2g$	Renewable forecasts
Line losses factor	$2n+1$	System

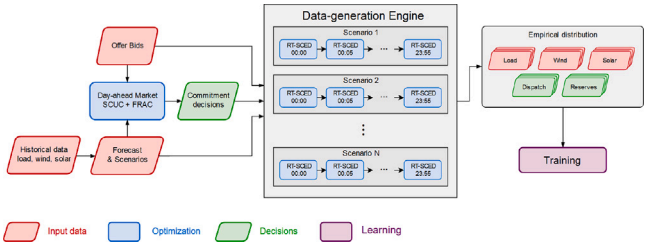

Fig. 5. Flow-chart of the Just-in-time Learning pipeline. Left: day-ahead market clearing. Middle: data generation using day-long simulations. Right: empirical distributions of load, wind, solar, and dispatch decisions are used as training data.

"classification then regression" approach, first screening the active generators to identifying those at their bounds, and then applying a regression to predict the dispatch of the remaining generators. The overall architecture is depicted in Fig. 4. The input of the learning task is a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i^c, \mathbf{y}_i^r)\}_{i=1}^N$, where $\mathbf{x}_i$, $\mathbf{y}_i^c$, $\mathbf{y}_i^r$ respectively represents the ith observation of the system state, the status indicators of the generators (i.e., whether each generator is at its maximum/minimum limits), and the optimal dispatches respectively.

The first architectural component classifies the active generators. Consider a power network with n buses, g generators and l loads. The classifier, parameterized by $\mathbf{w}_1$, is a mapping $C_{\mathbf{w}_1} : \mathbb{R}^d \to \{0,1\}^{2g}$, where d is the dimension of the input space of DNN model, that specifies whether a generator is at its upper limit or its lower limit, or neither. The input features describe the state of the power system and are detailed in Table 1. In the experiments presented in the subsequent section, the dimension d of the input space can be as large as 30,000. The output space of the classifier corresponds to the generator maximum/minimum limit constraints, where 1 indicates the dispatch of the generator is equal to the corresponding limit. Then, the objective of the learning task for the classifier is

$$\mathbf{w}_1^* = \arg\min_{\mathbf{w}_1} \sum_{i=1}^{N} \mathcal{L}_c(\mathbf{y}_i^c, C_{\mathbf{w}_1}(\mathbf{x}_i)) \tag{3}$$

where $\mathcal{L}_c$ denotes the cross entropy loss, i.e.,

$$\mathcal{L}_c(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{i=1}^{2g} \mathbf{y}_i \log(\hat{\mathbf{y}}_i) + (1-\mathbf{y}_i)\log(1-\hat{\mathbf{y}}_i).$$

The second architectural component, a regressor parameterized by $\mathbf{w}_2$, is a mapping $\mathcal{O}_{\mathbf{w}_2} : \mathbb{R}^{d+2n} \to \mathbb{R}^g$. The extra $2n$ dimensions come from the outputs of classifier, which are used to enforce the outputs of the regressor. Specifically, if the classifier predicts the 2nd generator at its minimum limits $\underline{g}$, the output of regressor corresponding to the 2nd generator will be set as $\underline{g}$. The objective of the learning task for the regression is

$$\mathbf{w}_2^* = \arg\min_{\mathbf{w}_2} \sum_{i=1}^{N} \mathcal{L}_o\left(\mathbf{y}_i^r, \mathcal{E}(\mathcal{O}_{\mathbf{w}_2}(\mathbf{x}_i), C_{\mathbf{w}_1^*}(\mathbf{x}_i))\right), \tag{4}$$

where $\mathcal{L}_o$ is the mean absolute error (MAE) loss, i.e., $\mathcal{L}_0(\mathbf{y}, \hat{\mathbf{y}}) = \|\mathbf{y} - \hat{\mathbf{y}}\|_1$, and $\mathcal{E}(.)$ ensures that the outputs of the regression respect the prediction of the classifier.

The regressor uses Latent Surgical Interventions (LSI) [30] as a building block to construct the regressor. LSI is a variant of residual networks [31] with interventions represented as binary vectors. As illustrated in Fig. 4, LSI enforces the interventions with a gated layer in the DNN architecture, allowing the DNN to be applicable to a range of configurations. As mentioned earlier, a significant factor of variability in the SCED comes from the combinatorial nature of the commitment decisions: two sets of commitment decisions can significantly change the optimal dispatches. The LSI architecture makes it possible to learn the generator dispatch for various commitment decisions, thereby allowing the proposed approach to generalize to the cases where the models are trained over multiple commitment decisions, e.g., when a ML model is trained for a time period longer than an hour.

3.3. Just-in-time learning pipeline

As discussed in Section 1.1, SCED instances have a lot of variability in real operations, and it is extremely challenging to learn the SCED optimization across so many possible commitment decisions and forecasts of loads and renewable energy sources. To mitigate this significant variability, this paper proposes a just-in-time learning pipeline that closely follows MISO's optimization pipeline. The machine-learning pipeline is depicted in Fig. 5 and consists of three phases: data preparation, training, and prediction.

Data preparation. At 12pm of day $D-1$, the TSO outputs the commitment decisions of generators for the next 24 h and Monte-Carlo scenarios for day D. Each scenario consists of 15 minute-level forecasts for loads and renewable generators. Since the SCED is solved every 5 min, the ML pipeline first linearly interpolates the forecasts at 5 min level. If necessary, load and renewable generations are further perturbing following the generation strategy of [9] to generator additional

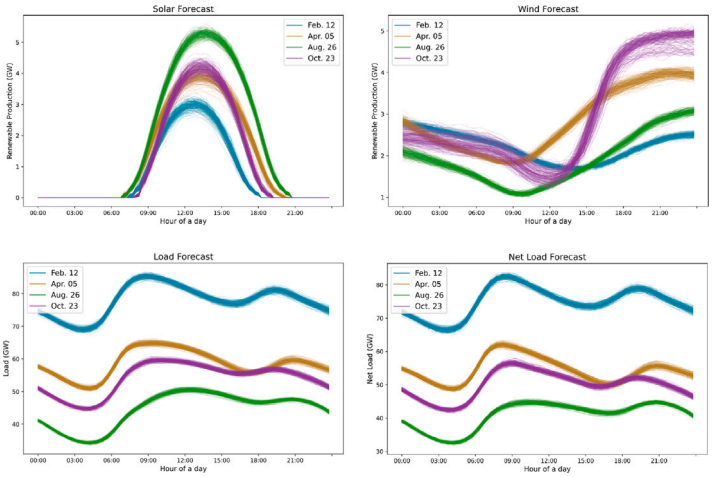

Fig. 6. Day-ahead scenarios for solar output (top-left), wind output (top-right), total load (bottom-left) and net load (bottom-right). 200 Monte-Carlo scenarios are depicted.

instances. The data is then used as input to SCED optimization models, which generate the corresponding optimal dispatches. The entire dataset is then divided into subsets, each of which spans one or a few hours, based on computational considerations. This data preparation process produces the inputs to the learning task described previously.

Training. For each subset, the pipeline first splits the overall dataset into the traditional training/validation/testing instances. It then trains the classifier and regressor in sequence, and the training step takes place in parallel for each subset.

Prediction. Starting from midnight on day D, at each time step, the corresponding ML model takes the latest system state as input and predicts the optimal dispatch of SCED models in real-time.

4. Experimental results

4.1. The test cases

The experiments replicate MISO's operations on the French transmission system, for four representative days in 2018, namely, February 12th, April 5th, August 26th, and October 23rd. These four days were selected at random to represent annual seasonality. For each operating day, 2000 Monte-Carlo scenarios for total load and renewable power production are sampled in a day-ahead fashion, i.e., scenarios are produced at noon of the previous day. These scenarios are illustrated in Fig. 6, which displays 200 scenarios for solar production, wind production, total load, and total net load, respectively. The total net load in this figure represents the amount of power production expected to be generated by the conventional generators, which is identical to the total load minus the renewable generation.

The forecasting models for load consumption and renewable power generation in this paper used Long Short-Term Memory (LSTM) neural networks [32] and the scenarios are generated by a MC-dropout approach [33]. Other forecasting methods and scenario generation approaches could be used instead, without changing the methodology.

Day-ahead commitment decisions are obtained by solving a DA-SCUC problem and recording its solution. The SCUC formulation used in the present experiment follows MISO's DA-SCUC described in [21, 34]. Again, the proposed ML pipeline is agnostic to the SCUC formulation itself and how it is solved, as it only requires the resulting commitments.

Finally, for each scenario and each day, 288 RT-SCED instances are solved (one every 5 min), yielding a total of 576,000 instances. This initial dataset is then divided into 24 hourly datasets, each containing 24,000 SCED instances, since commitment decisions are hourly. To avoid information leakage, the data is split between training/validation/testing instances as follows: 85% of scenarios are used for training, 7.5% for validation, and 7.5% for testing. All the reported results are in terms of the testing instances.

The optimization problems (SCUC and SCED) are formulated in the JuMP modeling language [35] and solved with Gurobi 9.1 [36], using Linux machines with dual Intel Xeon 6226@2.7 GHz CPUs on the PACE Phoenix cluster [37]; the entire data generation phase is performed in less than 2 h. The proposed CTR model is implemented using PyTorch [38] and trained using the Adam optimizer [39] with a learning rate of 5e−4. A grid search is performed to choose the hyperparameters, i.e., the dimension of the hidden layers (taken in the set {64, 128, 256}), and the number of layers (from the set {2, 3, 4, 5, 6}) for the fully connected layers in the LSI block. A leaky ReLU with leakiness of $\alpha = 0.01$ is used as the activation function in the LSI block. An early stopping criterion with 20 epochs is used for training the classifier and regressor models in order to prevent overfitting: when the loss values (Eqs. (3) and (4)) on the validation dataset do not decrease for 20 consecutive epochs, the training process is terminated. Training is performed using Tesla V100-PCIE GPUs with 16GBs HBM2 RAM, on machines with Intel CPU cores at 2.1 GHz.

4.2. Baselines

The proposed DNN-based CTR models are evaluated against the following baselines: Persistent CTR, Persistent Regression (Persistent Reg), and DDN-based Regression (Reg). The persistent baselines replicate the behavior of the previous SCED solution, i.e., they use the dispatch solution obtained 5 min earlier. The persistent baselines are motivated by the fact that, if the system does not change much between two consecutive intervals, then the SCED solution should not be changed much either. The Persistent CTR uses the same approach as the CTR models: it first predicts whether a given generator is at its minimum (resp., maximum) limit if it was at its minimum (resp. maximum) limit in the previous dispatch; then, for the remaining generators, it predicts the active dispatch using the regressor. The persistent baselines are expected to perform worse when large fluctuations in load and renewable production are observed, which typically occurs in the morning and evening. Note that these times of the day also display the largest ramping needs, and are among the most critical for reliability. The effectiveness of the CTR architecture is also demonstrated by comparing it to Regression alone for the optimal active dispatch, i.e., by omitting the classification step from the CTR.

Table 2
Average classification accuracy (%) of the Persistent and DNN classifiers on 4 representative days in 2018.

Methods	Date				Avg.
	Feb. 12	Apr. 5	Aug. 26	Oct. 23	
Persistent	98.26	97.79	98.64	98.31	98.25
DNN	99.56	99.18	99.40	99.24	99.35

Table 3
Evolution of the classification accuracy (in %) of the Persistent and DNN classifiers across the day.

Date	Method	4am	8am	12pm	4pm	8pm
Feb. 12	Persistent	98.79	98.66	97.96	98.71	99.04
	DNN	99.36	99.97	99.37	99.52	99.68
Apr. 05	Persistent	98.29	98.22	97.11	97.36	98.87
	DNN	99.34	99.31	98.46	98.93	99.37
Aug. 26	Persistent	99.73	98.94	98.08	97.85	98.42
	DNN	99.92	99.62	99.03	98.75	99.55
Oct. 23	Persistent	98.62	97.02	97.34	98.47	98.78
	DNN	99.55	97.02	98.61	99.31	99.48

4.3. Optimal dispatch prediction errors

Table 2 reports the overall classification accuracy of the Persistent and DNN classifiers across four representative days in 2018. Surprisingly, the persistent classifier is a strong baseline, with an accuracy that ranges from 97.79% in the Spring to 98.64% in the Summer. The DNN-based classifier always improves on the baseline, by around one percentage point on average and by up to 1.40 percentage point in the Spring. More detailed results about the classifiers are given in Table 3, which reports each classifier's accuracy across various hours of the day.

Next, Fig. 7 reports the mean absolute error (MAE) for active power dispatch of the different models for each of the considered days. Given a ground-truth dispatch $\mathbf{p}$ and the predicted dispatch $\hat{\mathbf{p}}$, the MAE is defined as

$$MAE = \frac{1}{N}\frac{1}{G}\sum_{i=1}^{N}\sum_{g=1}^{G}\|p_i^g - \hat{p}_i^g\|,$$

where N is the number of instances in the test dataset and G is the number of generators for each instance. As shown in Fig. 7, the MAEs of DNN-based CTR are always lower than those of DNN Regression, demonstrating the benefits of the classifier. The MAEs of Persistent CTR models are always lower than those of Persistent Regression alone, showing that even a simple classifier has benefits. Moreover, the methods using DNNs for classification and regression, i.e., CTR and Reg, are always better than their persistent counterparts, demonstrating the value of deep learning. Note that the performance of the persistent methods fluctuates during the day, mainly due to the variability of the commitments. In particular, persistent methods are not robust with respect to changes in commitments, contrary to the proposed DNN-based approach.

To investigate how the machine-learning models perform for different generator types, Table 4 reports their behavior for different generator sizes. The generators are clustered into three groups based on their actual active dispatches, and the MAE is reported for each group separately. Small-size generators have a capacity between 0 and 10MW, medium-size generators have a capacity between 10 and 100MW, and large-size generators have a capacity above 100MW. The last column report the MAEs across all generators.

Table 4 further confirms that the CTR models consistently outperform the corresponding regressors. These improvements are most notable for small and medium generators which are expected to have higher variability: the MAEs are decreased by up to 37.04% across small generators and 48.39% across medium generators (in Aug. 26). Moreover, across four days, the CTR model achieves a Mean Average Percentage Error (MAPE) of 0.59% and 0.34% for medium and large generators.

Table 4
Mean absolute error (MW) by generator size[†].

Date	Method	Small	Medium	Large	All
Feb. 12	Persistent Reg	0.122	0.465	1.602	0.374
	Persistent CTR	0.084	0.333	1.128	0.262
	DNN Reg	0.057	0.188	0.654	0.153
	DNN CTR	**0.043**	**0.141**	**0.535**	**0.117**
Apr. 05	Persistent Reg	0.242	0.345	7.480	0.772
	Persistent CTR	0.197	0.220	4.374	0.463
	DNN Reg	0.149	0.152	2.553	0.282
	DNN CTR	**0.105**	**0.110**	**2.291**	**0.241**
Aug. 26	Persistent Reg	0.097	0.256	7.447	0.637
	Persistent CTR	0.080	0.149	4.045	0.352
	DNN Reg	0.054	0.124	2.454	0.218
	DNN CTR	**0.034**	**0.064**	**2.220**	**0.190**
Oct. 23	Persistent Reg	0.176	0.535	8.425	0.778
	Persistent CTR	0.140	0.341	4.596	0.434
	DNN Reg	0.106	0.192	2.724	0.263
	DNN CTR	**0.076**	**0.145**	**2.525**	**0.235**

[†]Small: 0–10 MW; Medium: 10–100 MW; Large: >100 MW

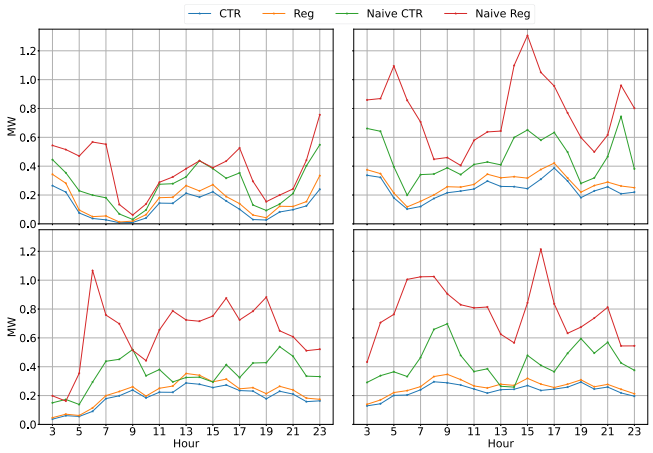

Fig. 7. Mean Absolute Error (MAE) Over Time on Feb. 12 (top-left), Apr. 5 (top-right), Aug. 26 (bottom-left), and Oct. 23 (bottom-right).

4.4. Solving time vs inference time

Finally, the benefits of the optimization proxy in terms of computing times are quantified. First, solving the SCED using traditional optimization tools takes, on average, 15.93 s. Actual computing times fluctuate throughout the day, with hours of increased load and congestion leading to higher computing times. In contrast, evaluating the optimization proxy for a batch of 288 instances takes an average 1.5 ms. In other words, roughly 200,000 scenarios may be evaluated in less than one second. This represents an improvement of 4 orders of magnitude, even under the assumption that several hundred SCED instances can be solved in parallel.

5. Conclusion

The paper has presented several challenges that may hamper the use of ML models in power systems operations, namely, the variability of electricity demand and renewable production, the variability of generators' production costs, and the combinatorial structure of commitment decisions. To address these challenges, a novel ML pipeline has been proposed, which leverages day-ahead forecasts and a TSO's knowledge of commitment decisions several hours before they come into effect.

The proposed pipeline consists of three phases: (1) data preparation, (2) training, and (3) real-time predictions, which fit naturally in a TSO's operations. Computational experiments have been carried by replicating MISO's operational pipeline on a real, large-scale transmission system, and demonstrate the feasibility of the approach. Informed by the behavior of real-time markets, a novel combined classifier+regressor architecture has been proposed, which has been shown to improve model accuracy. Numerical results indicate that optimization proxies using ML models have the potential to provide operators with new tools for real-time risk monitoring.

Several extensions are possible, which will be the topic of future work. The integration, during training, of Lagrangian-based penalties has the potential to further improve the performance of the DNN models presented here. Such methods have been shown to improve the predicted solution's feasibility with respect to the optimization problem's original constraints. Moreover, once it is trained, the proposed classifier may be used to also accelerate the SCED resolution, by providing hints as to which variables should be fixed at their minimum or maximum limit. Finally, the combined classification+regression architecture is not limited to SCED models, and may be applied to any optimization problem. Its application to unit-commitment problems will be considered in future extensions of this work.

CRediT authorship contribution statement

Wenbo Chen: Conceptualization, Methodology, Software, Validation, Analysis, Investigation, Data curation, Visualization, Writing. **Seonho Park:** Conceptualization, Methodology, Software, Validation, Analysis, Investigation, Data curation, Visualization, Writing. **Mathieu Tanneau:** Conceptualization, Methodology, Software, Validation, Analysis, Investigation, Data curation, Writing, Supervision. **Pascal Van Hentenryck:** Conceptualization, Methodology, Validation, Analysis, Writing, Supervision, Funding.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is partly funded by NSF Award 1912244 and 2112533, and ARPA-E Perform Award AR0001136.

References

[1] A.J. Conejo, L. Baringo, Power System Operations, Springer, 2018.
[2] Y. Chen, F. Wang, J. Wan, F. Pan, Developing next generation electricity market clearing optimization software, in: 2018 IEEE Power Energy Society General Meeting, PESGM, 2018, pp. 1–5, http://dx.doi.org/10.1109/PESGM.2018.8586548.
[3] N. Chiang, A. Grothey, Solving security constrained optimal power flow problems by a structure exploiting interior point method, Opt. Eng. 16 (1) (2015) 49–71.
[4] Q. Wang, J.D. McCalley, T. Zheng, E. Litvinov, Solving corrective risk-based security-constrained optimal power flow with Lagrangian relaxation and Benders decomposition, Int. J. Electr. Power Energy Syst. 75 (2016) 255–264, http://dx.doi.org/10.1016/j.ijepes.2015.09.001.
[5] A. Velloso, P. Van Hentenryck, E.S. Johnson, An exact and scalable problem decomposition for security-constrained optimal power flow, Electr. Power Syst. Res. 195 (2021) 106677.
[6] T. Werho, J. Zhang, V. Vittal, Y. Chen, A. Thatte, L. Zhao, Scenario generation of wind farm power for real-time system operation, 2021, arXiv:2106.09105.
[7] X. Pan, T. Zhao, M. Chen, DeepOPF: Deep neural network for DC optimal power flow, in: 2019 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm), IEEE, 2019, pp. 1–6.
[8] F. Fioretto, T.W. Mak, P. Van Hentenryck, Predicting AC optimal power flows: Combining deep learning and lagrangian dual methods, Proc. AAAI Conf. Artif. Intell. 34 (01) (2020) 630–637.
[9] M. Chatzos, F. Fioretto, T.W. Mak, P. Van Hentenryck, High-fidelity machine learning approximations of large-scale optimal power flow, 2020, arXiv preprint arXiv:2006.16356.
[10] X. Lei, Z. Yang, J. Yu, J. Zhao, Q. Gao, H. Yu, Data-driven optimal power flow: A physics-informed machine learning approach, IEEE Trans. Power Syst. 36 (1) (2020) 346–354.
[11] M. Chatzos, T.W.K. Mak, P.V. Hentenryck, Spatial network decomposition for fast and scalable AC-OPF learning, IEEE Transactions on Power Systems 37 (4) (2022) 2601–2612, http://dx.doi.org/10.1109/TPWRS.2021.3124726.
[12] A. Velloso, P. Van Hentenryck, Combining deep learning and optimization for preventive security-constrained DC optimal power flow, IEEE Transactions on Power Systems 36 (4) (2021) 3618–3628, http://dx.doi.org/10.1109/TPWRS.2021.3054341.
[13] A.S. Zamzam, K. Baker, Learning optimal solutions for extremely fast AC optimal power flow, in: 2020 IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm), IEEE, 2020, pp. 1–6.
[14] D. Owerko, F. Gama, A. Ribeiro, Optimal power flow using graph neural networks, in: ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE, 2020, pp. 5930–5934.
[15] D. Deka, S. Misra, Learning for DC-OPF: Classifying active sets using neural nets, in: 2019 IEEE Milan PowerTech, IEEE, 2019, pp. 1–6.
[16] S. Misra, L. Roald, Y. Ng, Learning for constrained optimization: Identifying optimal active constraint sets, INFORMS J. Comput. (2021).
[17] A.S. Xavier, F. Qiu, S. Ahmed, Learning to solve large-scale security-constrained unit commitment problems, INFORMS J. Comput. 33 (2) (2021) 739–756.
[18] Y. Yang, Z. Yang, J. Yu, K. Xie, L. Jin, Fast economic dispatch in smart grids using deep learning: An active constraint screening approach, IEEE Internet Things J. 7 (11) (2020) 11030–11040.
[19] N. Guha, Z. Wang, M. Wytock, A. Majumdar, Machine learning for AC optimal power flow, 2019, arXiv preprint arXiv:1910.08842.
[20] A. Venzke, D.T. Viola, J. Mermet-Guyennet, G.S. Misyris, S. Chatzivasileiadis, Neural networks for encoding dynamic security-constrained optimal power flow to mixed-integer linear programs, 2020, arXiv preprint arXiv:2003.07939.
[21] MISO, Energy and Operating Reserve Markets, Business Practices Manual Energy and Operating Reserve Markets.
[22] CAISO, Business Practice Manual for Market Operations.
[23] ISO-NE, Market Operations. Manual M-11.
[24] PJM, PJM Manual 11: Energy and Ancillary Services Market Operations.
[25] NYISO, Day-Ahead Scheduling Manual Manual 11.
[26] MISO, Real-Time Energy and Operating Reserve Market Software Formulations and Business Logic, Business Practices Manual Energy and Operating Reserve Markets Attachment D.
[27] X. Ma, Y. Chen, J. Wan, Midwest ISO co-optimization based real-time dispatch and pricing of energy and ancillary services, in: 2009 IEEE Power Energy Society General Meeting, 2009, pp. 1–6, http://dx.doi.org/10.1109/PES.2009.5276010.
[28] B. Eldridge, R.P. O'Neill, A. Castillo, Marginal loss calculations for the DCOPF, 2017.
[29] M. Chatzos, M. Tanneau, P. Van Hentenryck, Data-driven time series reconstruction for modern power systems research, Electric Power Systems Research 212 (2022) 108589, http://dx.doi.org/10.1016/j.epsr.2022.108589.
[30] B. Donnot, I. Guyon, Z. Liu, M. Schoenauer, A. Marot, P. Panciatici, Latent surgical interventions in residual neural networks, 2018.
[31] K. He, X. Zhang, S. Ren, J. Sun, Identity mappings in deep residual networks, in: European Conference on Computer Vision, Springer, 2016, pp. 630–645.
[32] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[33] Y. Gal, Z. Ghahramani, Dropout as a bayesian approximation: Representing model uncertainty in deep learning, in: International Conference on Machine Learning, PMLR, 2016, pp. 1050–1059.
[34] MISO, Day-Ahead Energy and Operating Reserve Market Software Formulations and Business Logic, Business Practices Manual Energy and Operating Reserve Markets Attachment B.
[35] I. Dunning, J. Huchette, M. Lubin, JuMP: A modeling language for mathematical optimization, SIAM Rev. 59 (2) (2017) 295–320, http://dx.doi.org/10.1137/15M1020575.
[36] L. Gurobi Optimization, Gurobi optimizer reference manual, 2021, URL https://www.gurobi.com.
[37] PACE, Partnership for an advanced computing environment (PACE), 2017, URL http://www.pace.gatech.edu.
[38] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, Pytorch: An imperative style, high-performance deep learning library, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. dÁlché Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems 32, Curran Associates, Inc., 2019, pp. 8024–8035.
[39] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.